Experiment 11: 180-Degree Rotation Clockwise

Aim: To perform a 180-degree clockwise rotation on the given input image using Python and OpenCV.

Procedure

1. Import the cv2 library.
2. Read the source image from the given file path using cv2.imread().
3. Apply 180-degree rotation using cv2.rotate() with flag cv2.ROTATE_180.
4. Display the original and rotated images in separate output windows.
5. Pause execution using cv2.waitKey(0) until a key is pressed.
6. Destroy all OpenCV display windows using cv2.destroyAllWindows().

Code

```
import cv2

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
rotated180 = cv2.rotate(img, cv2.ROTATE_180)

cv2.imshow("Original Image", img)
cv2.imshow("180 Degree Rotated", rotated180)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The given image was successfully rotated 180 degrees clockwise using Python OpenCV.

Experiment 12: 270-Degree Rotation Clockwise

Aim: To perform a 270-degree clockwise rotation on the given input image using Python and OpenCV.

Procedure

1. Import the cv2 library.
2. Read the source image from the given file path using cv2.imread().
3. Apply 270-degree clockwise rotation using cv2.rotate() with flag cv2.ROTATE_90_COUNTERCLOCKWISE.
4. Display the original and rotated images in separate output windows.
5. Pause execution using cv2.waitKey(0) until a key is pressed.
6. Destroy all OpenCV display windows using cv2.destroyAllWindows().

Code

```
import cv2

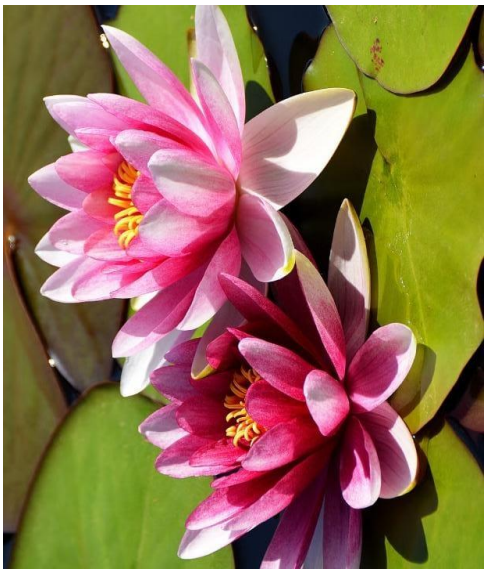
img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
rotated270 = cv2.rotate(img, cv2.ROTATE_90_COUNTERCLOCKWISE)

cv2.imshow("Original Image", img)
cv2.imshow("270 Degree Rotated", rotated270)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The given image was successfully rotated 270 degrees clockwise using Python OpenCV.

Experiment 13: Affine Transformation

Aim: To perform an affine transformation on the given input image using Python and OpenCV.

Procedure

1. Import the required cv2 and numpy libraries.
2. Load the input image using cv2.imread() and read its height and width.
3. Define three source points and the three corresponding destination points using np.float32().
4. Compute the 2×3 transformation matrix using cv2.getAffineTransform().
5. Apply the transformation to the image using cv2.warpAffine().
6. Display the original and transformed images and close all windows on key press.

Code

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
rows, cols = img.shape[:2]

src = np.float32([[0, 0], [cols - 1, 0], [0, rows - 1]])
dst = np.float32([[cols * 0.1, rows * 0.1], [cols * 0.9, rows * 0.2], [cols * 0.2, rows * 0.9]])

M = cv2.getAffineTransform(src, dst)
affine = cv2.warpAffine(img, M, (cols, rows))

cv2.imshow("Original Image", img)
cv2.imshow("Affine Transformed Image", affine)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The affine transformation was successfully applied to the input image using OpenCV.

Experiment 14: Perspective Transformation

Aim: To perform a perspective transformation on the given input image using Python and OpenCV.

Procedure

1. Import the required cv2 and numpy libraries.
2. Load the input image using cv2.imread() and read its height and width.
3. Define four source corner points and four destination points using np.float32().
4. Compute the 3×3 perspective matrix using cv2.getPerspectiveTransform().
5. Apply the transformation to the image using cv2.warpPerspective().
6. Display the original and transformed images and close all windows on key press.

Code

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
rows, cols = img.shape[:2]

src = np.float32([[0, 0], [cols - 1, 0], [0, rows - 1], [cols - 1, rows - 1]])
dst = np.float32([[cols * 0.15, rows * 0.1], [cols * 0.85, 0], [0, rows - 1], [cols - 1, rows * 0.9]])
```

```
M = cv2.getPerspectiveTransform(src, dst)
perspective = cv2.warpPerspective(img, M, (cols, rows))

cv2.imshow("Original Image", img)
cv2.imshow("Perspective Transformed Image", perspective)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The perspective transformation was successfully applied to the input image using OpenCV.

Experiment 15: Harris Corner Detection

Aim: To read an input image using Python OpenCV and detect the corners in the image using the Harris Corner Detection function.

Procedure

1. Import the required cv2 and numpy libraries.
2. Load the input image using cv2.imread() and convert it to gray-scale using cv2.cvtColor().
3. Convert the gray-scale image to float32 format using np.float32().
4. Detect corners using cv2.cornerHarris() with block size, aperture size and k value.
5. Dilate the result and mark the pixels above a threshold in red on a copy of the image.
6. Display the original and corner-marked images and close all windows on key press.

Code

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = np.float32(gray)

dst = cv2.cornerHarris(gray, 2, 3, 0.04)
dst = cv2.dilate(dst, np.ones((3, 3), np.uint8))

corners = img.copy()
corners[dst > 0.01 * dst.max()] = [0, 255, 0]

cv2.imshow("Original Image", img)
cv2.imshow("Harris Corners", corners)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The corners in the input image were successfully detected and marked using OpenCV's Harris Corner Detection function.

Experiment 16: Sobel Edge Filtering

Aim: To implement the Sobel algorithm using OpenCV to filter the input image.

Procedure

1. Import the cv2 library.
2. Load the input image using cv2.imread() and convert it to gray-scale.
3. Compute the horizontal gradient using cv2.Sobel() with dx = 1 and dy = 0.
4. Compute the vertical gradient using cv2.Sobel() with dx = 0 and dy = 1.
5. Convert both gradients to 8-bit images using cv2.convertScaleAbs() and combine them using cv2.addWeighted().
6. Display the original, Sobel X, Sobel Y and combined images and close all windows on key press.

Code

```
import cv2

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

sobelx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

sobelx = cv2.convertScaleAbs(sobelx)
sobely = cv2.convertScaleAbs(sobely)
sobel = cv2.addWeighted(sobelx, 0.5, sobely, 0.5, 0)

cv2.imshow("Original Image", img)
cv2.imshow("Sobel X", sobelx)
cv2.imshow("Sobel Y", sobely)
cv2.imshow("Sobel Combined", sobel)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output





Result: The input image was successfully filtered using the Sobel algorithm in OpenCV.

Experiment 17: Image Watermarking

Aim: To design and implement a watermarking technique to insert a watermark into the original image using OpenCV.

Procedure

1. Import the cv2 library.
2. Load the input image using cv2.imread() and make a copy of it as an overlay.
3. Write the watermark text on the overlay using cv2.putText().
4. Blend the overlay with the original image using cv2.addWeighted() to make the watermark semi-transparent.
5. Display the original and watermarked images.
6. Wait for a key press and close all display windows using cv2.destroyAllWindows().

Code

```
import cv2

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
overlay = img.copy()
cv2.putText(overlay, "SIMATS", (30, img.shape[0] - 40), cv2.FONT_HERSHEY_SIMPLEX, 2, (255, 255, 255), 5)

watermarked = cv2.addWeighted(overlay, 0.5, img, 0.5, 0)
```

```
cv2.imshow("Original Image", img)
cv2.imshow("Watermarked Image", watermarked)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The watermark was successfully inserted into the original image using OpenCV.

Experiment 18: Image Cropping, Copying and Pasting (ROI)

Aim: To implement image cropping, copying and pasting to select a region of interest (ROI) from the source image using OpenCV.

Procedure

1. Import the cv2 library.
2. Load the input image using cv2.imread().
3. Select the region of interest (ROI) by slicing the image array using row and column ranges.
4. Copy the selected ROI using the copy() function.
5. Paste the copied ROI at another location of the image using array slicing.
6. Display the original, cropped and pasted images and close all windows on key press.

Code

```
import cv2
```

```
img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
roi = img[200:350, 380:560]
copied = roi.copy()

pasted = img.copy()
pasted[20:170, 20:200] = copied

cv2.imshow("Original Image", img)
cv2.imshow("Cropped ROI", roi)
cv2.imshow("Pasted ROI", pasted)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output





Result: The region of interest was successfully cropped, copied and pasted on the source image using OpenCV.

Experiment 19: Erosion (Morphological Operation)

Aim: To implement the Erosion technique as a morphological operation to shrink the foreground regions using OpenCV in Python.

Procedure

1. Import the required cv2 and numpy libraries.
2. Load the input image using cv2.imread() and convert it to gray-scale.

3. Convert the gray-scale image to a binary image using cv2.threshold().
4. Define a 5×5 structuring element (kernel) using np.ones().
5. Apply the erosion operation using cv2.morphologyEx() with cv2.MORPH_ERODE.
6. Display the binary and result images and close all windows on key press.

Code

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

kernel = np.ones((5, 5), np.uint8)
result = cv2.morphologyEx(binary, cv2.MORPH_ERODE, kernel)

cv2.imshow("Binary Image", binary)
cv2.imshow("Erosion Image", result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output





Result: The erosion morphological operation was successfully applied to the input image using OpenCV.

Experiment 20: Dilation (Morphological Operation)

Aim: To implement the Dilation technique as a morphological operation to dilate the foreground regions using OpenCV in Python.

Procedure

1. Import the required cv2 and numpy libraries.
2. Load the input image using cv2.imread() and convert it to gray-scale.
3. Convert the gray-scale image to a binary image using cv2.threshold().
4. Define a 5×5 structuring element (kernel) using np.ones().
5. Apply the dilation operation using cv2.morphologyEx() with cv2.MORPH_DILATE.
6. Display the binary and result images and close all windows on key press.

Code

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

kernel = np.ones((5, 5), np.uint8)
result = cv2.morphologyEx(binary, cv2.MORPH_DILATE, kernel)

cv2.imshow("Binary Image", binary)
cv2.imshow("Dilation Image", result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The dilation morphological operation was successfully applied to the input image using OpenCV.

Experiment 21: Opening (Morphological Operation)

Aim: To implement the Opening technique as a morphological operation (erosion followed by dilation) to remove small noise from the foreground using OpenCV in Python.

Procedure

1. Import the required cv2 and numpy libraries.
2. Load the input image using cv2.imread() and convert it to gray-scale.
3. Convert the gray-scale image to a binary image using cv2.threshold().
4. Define a 5×5 structuring element (kernel) using np.ones().
5. Apply the opening operation using cv2.morphologyEx() with cv2.MORPH_OPEN.
6. Display the binary and result images and close all windows on key press.

Code

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

kernel = np.ones((5, 5), np.uint8)
result = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)

cv2.imshow("Binary Image", binary)
cv2.imshow("Opening Image", result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output





Result: The opening morphological operation was successfully applied to the input image using OpenCV.

Experiment 22: Closing (Morphological Operation)

Aim: To implement the Closing technique as a morphological operation (dilation followed by erosion) to close small holes in the foreground using OpenCV in Python.

Procedure

1. Import the required cv2 and numpy libraries.
2. Load the input image using cv2.imread() and convert it to gray-scale.
3. Convert the gray-scale image to a binary image using cv2.threshold().
4. Define a 5×5 structuring element (kernel) using np.ones().
5. Apply the closing operation using cv2.morphologyEx() with cv2.MORPH_CLOSE.
6. Display the binary and result images and close all windows on key press.

Code

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
```

```
kernel = np.ones((5, 5), np.uint8)
result = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

cv2.imshow("Binary Image", binary)
cv2.imshow("Closing Image", result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The closing morphological operation was successfully applied to the input image using OpenCV.

Experiment 23: Top Hat (Morphological Operation)

Aim: To implement the Top Hat technique as a morphological operation (difference between the input and its opening) to extract small bright regions using OpenCV in Python.

Procedure

1. Import the required cv2 and numpy libraries.
2. Load the input image using cv2.imread() and convert it to gray-scale.
3. Define a 15×15 elliptical structuring element using cv2.getStructuringElement().
4. Apply the top hat operation using cv2.morphologyEx() with cv2.MORPH_TOPHAT.
5. Display the gray-scale and result images.
6. Wait for a key press and close all display windows using cv2.destroyAllWindows().

Code

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (15, 15))
result = cv2.morphologyEx(gray, cv2.MORPH_TOPHAT, kernel)

cv2.imshow("Gray Image", gray)
cv2.imshow("Top Hat Image", result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output





Result: The top hat morphological operation was successfully applied to the input image using OpenCV.

Experiment 24: Black Hat (Morphological Operation)

Aim: To implement the Black Hat technique as a morphological operation (difference between the closing and the input) to extract small dark regions using OpenCV in Python.

Procedure

1. Import the required cv2 and numpy libraries.
2. Load the input image using cv2.imread() and convert it to gray-scale.

3. Define a 15×15 elliptical structuring element using cv2.getStructuringElement().
4. Apply the black hat operation using cv2.morphologyEx() with cv2.MORPH_BLACKHAT.
5. Display the gray-scale and result images.
6. Wait for a key press and close all display windows using cv2.destroyAllWindows().

Code

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (15, 15))
result = cv2.morphologyEx(gray, cv2.MORPH_BLACKHAT, kernel)

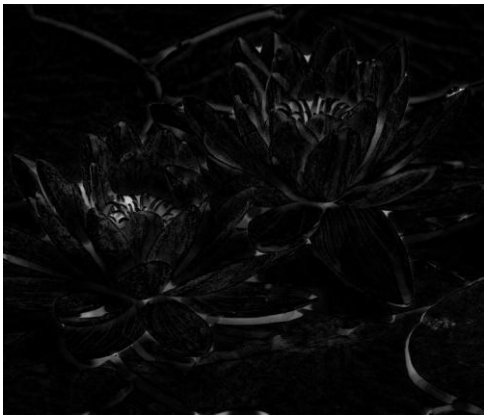
cv2.imshow("Gray Image", gray)
cv2.imshow("Black Hat Image", result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output





Result: The black hat morphological operation was successfully applied to the input image using OpenCV.

Experiment 25: Watch Recognition (General Object Recognition)

Aim: To recognize a watch from the given image by general object recognition using OpenCV.

Procedure

1. Import the required cv2 and numpy libraries.
2. Load the input image using cv2.imread() and convert it to gray-scale.
3. Smooth the gray-scale image using cv2.medianBlur() to reduce noise.
4. Detect the circular watch dial using cv2.HoughCircles().
5. Draw a bounding box and the label "Watch" around the detected object using cv2.rectangle() and cv2.putText().
6. Display the original and recognized images and close all windows on key press.

Code

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\acer\Downloads\watch.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = cv2.medianBlur(gray, 5)

circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, 1, 200, param1=100, param2=40,
minRadius=100, maxRadius=200)

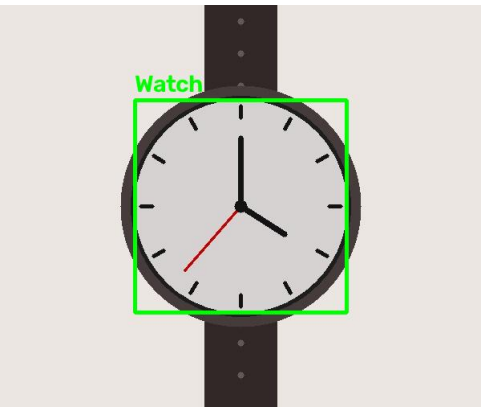
output = img.copy()
if circles is not None:
    for x, y, r in np.uint16(np.around(circles[0])):
        cv2.rectangle(output, (x - r, y - r), (x + r, y + r), (0, 255, 0), 3)
        cv2.putText(output, "Watch", (x - r, y - r - 10), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

cv2.imshow("Original Image", img)
cv2.imshow("Recognized Watch", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The watch was successfully recognized in the given image using OpenCV object recognition.

Experiment 26: Reverse the Frames of a Video

Aim: To implement a function to reverse the frames of a video to create a video in reverse mode using OpenCV.

Procedure

1. Import the required cv2 and numpy libraries.
2. Load the image and create a sequence of video frames by progressively zooming into it.
3. Reverse the order of the frames using list slicing (frames[::-1]).
4. Write the reversed frames to a new video file using cv2.VideoWriter().
5. Display sample frames of the forward and the reversed sequence.
6. Wait for a key press and close all display windows using cv2.destroyAllWindows().

Code

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\acer\Downloads\pretty lotus.jpeg")
h, w = img.shape[:2]

frames = []
for i in range(20):
    s = 0.5 * i / 19
    crop = img[int(h * s / 2):h - int(h * s / 2), int(w * s / 2):w - int(w * s / 2)]
    frames.append(cv2.resize(crop, (w, h)))

reversed_frames = frames[::-1]

out = cv2.VideoWriter("reversed.avi", cv2.VideoWriter_fourcc(*"XVID"), 10, (w, h))
for frame in reversed_frames:
    out.write(frame)
out.release()

forward = np.hstack([cv2.resize(f, (180, 154)) for f in frames[::6]])
backward = np.hstack([cv2.resize(f, (180, 154)) for f in reversed_frames[::6]])
```

```
cv2.imshow("Forward Frames", forward)
cv2.imshow("Reversed Frames", backward)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The frames of the video were successfully reversed and written as a reverse-mode video using OpenCV.

Experiment 27: Face Detection

Aim: To implement a face detection algorithm using OpenCV to detect and locate human faces in an image.

Procedure

1. Import the cv2 library.
2. Load the Haar cascade classifier for frontal faces using cv2.CascadeClassifier().
3. Load the input image using cv2.imread() and convert it to gray-scale.
4. Detect faces using detectMultiScale() with scale factor and minimum neighbours.
5. Draw a rectangle around every detected face using cv2.rectangle().
6. Display the original and the detected images and close all windows on key press.

Code

```
import cv2

img = cv2.imread(r"C:\Users\acer\Downloads\face.jpg")
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + "haarcascade_frontalface_default.xml")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)

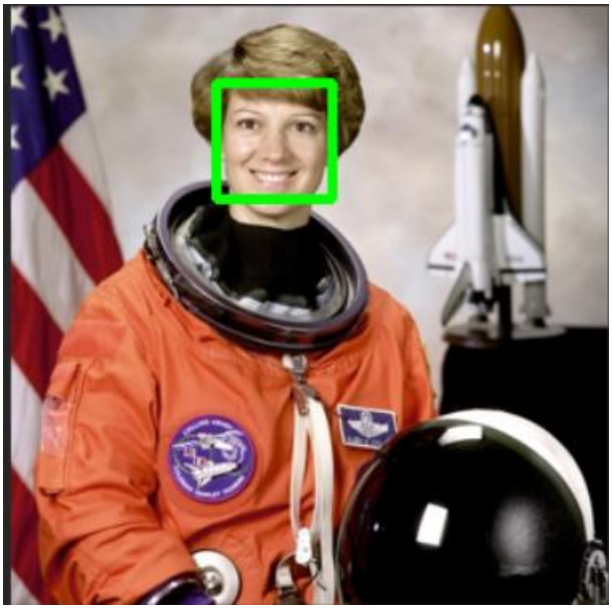
output = img.copy()
for (x, y, w, h) in faces:
    cv2.rectangle(output, (x, y), (x + w, y + h), (0, 255, 0), 3)

cv2.imshow("Original Image", img)
cv2.imshow("Face Detection", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The human face in the given image was successfully detected and located using OpenCV.

Experiment 28: Vehicle Detection

Aim: To implement a vehicle detection algorithm using OpenCV to detect and locate vehicles in each frame of a video.

Procedure

1. Import the required cv2 and numpy libraries.
2. Load the frame using cv2.imread() and split it into its B, G and R channels.
3. Detect the edges of every channel using cv2.Canny(), combine them using cv2.bitwise_or() and close the gaps using cv2.morphologyEx().
4. Find the object contours using cv2.findContours().
5. Filter the contours by area and width-to-height ratio and draw a bounding box with the label "Vehicle".
6. Display the original and the detected frames and close all windows on key press.

Code

```
import cv2
import numpy as np

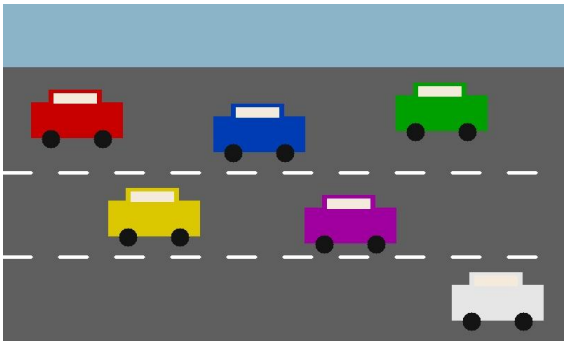
img = cv2.imread(r"C:\Users\acer\Downloads\traffic.jpg")
edges = np.zeros(img.shape[:2], np.uint8)
for channel in cv2.split(img):
    edges = cv2.bitwise_or(edges, cv2.Canny(channel, 50, 150))
edges = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, np.ones((9, 9), np.uint8))

contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

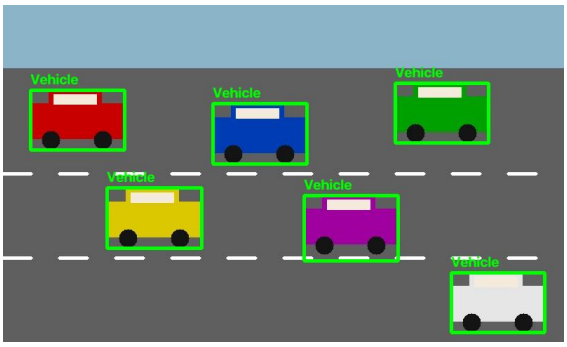
output = img.copy()
for c in contours:
    x, y, w, h = cv2.boundingRect(c)
    if w * h > 5000 and 1.3 < w / h < 3.5 and y > 80:
        cv2.rectangle(output, (x, y), (x + w, y + h), (0, 255, 0), 3)
        cv2.putText(output, "Vehicle", (x, y - 8), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

cv2.imshow("Original Frame", img)
cv2.imshow("Vehicle Detection", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The vehicles in the given frame were successfully detected and located using OpenCV.

Experiment 29: Eye Detection

Aim: To implement an eye detection algorithm using OpenCV to detect and locate human eyes in an image.

Procedure

1. Import the cv2 library.
2. Load the Haar cascade classifiers for faces and eyes using cv2.CascadeClassifier().
3. Load the input image using cv2.imread() and convert it to gray-scale.
4. Detect the faces and, inside each face region, detect the eyes using detectMultiScale().
5. Draw a rectangle around every detected eye using cv2.rectangle().
6. Display the original and the detected images and close all windows on key press.

Code

```
import cv2

img = cv2.imread(r"C:\Users\acer\Downloads\face.jpg")
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
eye_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + "haarcascade_eye.xml")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, 1.1, 5)

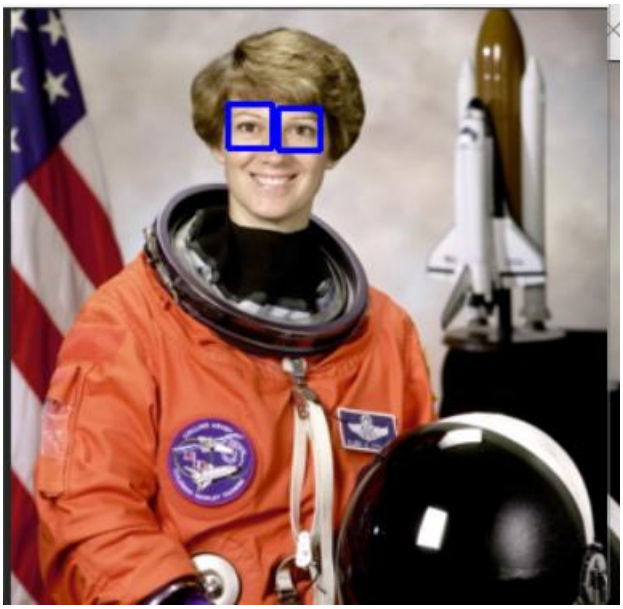
output = img.copy()
for (x, y, w, h) in faces:
    roi_gray = gray[y:y + h, x:x + w]
    roi_color = output[y:y + h, x:x + w]
    for (ex, ey, ew, eh) in eye_cascade.detectMultiScale(roi_gray):
        cv2.rectangle(roi_color, (ex, ey), (ex + ew, ey + eh), (255, 0, 0), 2)

cv2.imshow("Original Image", img)
cv2.imshow("Eye Detection", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output



Result: The human eyes in the given image were successfully detected and located using OpenCV.

Experiment 30: Smile Detection

Aim: To implement a smile detection algorithm using OpenCV to detect and locate a human smile in an image.

Procedure

1. Import the cv2 library.
2. Load the Haar cascade classifiers for faces and smiles using cv2.CascadeClassifier().
3. Load the input image using cv2.imread() and convert it to gray-scale.
4. Detect the faces and, inside each face region, detect the smile using detectMultiScale().
5. Draw a rectangle around every detected smile using cv2.rectangle().
6. Display the original and the detected images and close all windows on key press.

Code

```
import cv2

img = cv2.imread(r"C:\Users\acer\Downloads\face.jpg")
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
smile_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + "haarcascade_smile.xml")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, 1.1, 5)

output = img.copy()
for (x, y, w, h) in faces:
    roi_gray = gray[y:y + h, x:x + w]
    roi_color = output[y:y + h, x:x + w]
    for (sx, sy, sw, sh) in smile_cascade.detectMultiScale(roi_gray, 1.7, 20):
        cv2.rectangle(roi_color, (sx, sy), (sx + sw, sy + sh), (0, 0, 255), 2)

cv2.imshow("Original Image", img)
cv2.imshow("Smile Detection", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input



Output:



Result: The human smile in the given image was successfully detected and located using OpenCV.